ONG CONNECT

Documentação do projeto

Modelagem híbrida relacional e documental com JSON no MariaDB

Modelagem de Banco de Dados • Apresentação prevista: 27/10/2026

Integrantes do grupo

Nome	RGM
Daniel Costa Ferreira	048098108
Daniel Paulino	048711152
Dioni Alves Moreira	047399112
Igor Roehr lúcio de Godoy Strauss	5947848251
Luiz Carlos Domeneguetti Lourenção	047147695
Luiz Paulo Gomes do Prado e Silva	048424170
Raphael Vital	047147768

Turma: Modelagem de Bancos de Dados — terça-feira, noite. Instituição: Cruzeiro do Sul — Campus Paulista. O grupo possui sete integrantes, conforme o limite do enunciado.

Escopo: documentação acadêmica e demonstração com dados fictícios. A aceitação do recorte como tecnologia emergente deve ser confirmada com o professor; este documento não declara aprovação ou envio da atividade.

Sumário

1. Estudo acadêmico 3

2. Modelo de dados e relacionamentos 7

3. Experimento e resultados 16

4. Requisitos e permissões 19

5. Casos de uso 20

6. Arquitetura 21

7. Instalação e operação 22

8. Referências 24

9. Matriz de conformidade 25

10. Roteiro de apresentação 27

11. Segurança e melhorias para operação online 29

1. Estudo acadêmico

Resumo

O trabalho estuda o uso de documentos JSON junto a entidades relacionais no MariaDB, utilizando um portal de organizações sociais como caso prático. O domínio contém contas, ONGs, projetos, oportunidades, candidaturas e recebimentos simulados. Os identificadores, vínculos, estados e valores financeiros são modelados em colunas e relações; alguns atributos variáveis e listas ficam em documentos JSON. A comparação usa uma representação tradicional com uma tabela de dias por oportunidade e uma representação documental com a lista no JSON. Um experimento local reproduzível compara equivalência das consultas, extensão do cadastro e validação sintática. O resultado sustenta a flexibilidade do documento para esse exemplo, sem demonstrar superioridade geral de desempenho. O DER e os scripts permitem relacionar a pesquisa à implementação real.

Palavras-chave: modelagem de dados; MariaDB; JSON; integridade referencial; dados semiestruturados.

1. Problema e objetivo

Oportunidades de voluntariado podem possuir modalidade, local, vários dias de atuação, horários e orientações distintas. A identidade da oportunidade e sua ligação ao projeto são estáveis; parte do cadastro pode evoluir. Como representar essa variedade sem perder os vínculos com projetos e candidaturas?

O objetivo geral é avaliar uma modelagem híbrida de dados relacionais e documentos JSON no contexto do ONG Connect. Os objetivos específicos são descrever fundamentos, explicitar benefícios e custos, apresentar o modelo relacional real, construir um exemplo comparável e interpretar os resultados com limitações.

O enunciado apresenta uma loja como exemplo. Este trabalho utiliza um domínio diferente com entidades, cardinalidades e integridade equivalentes em finalidade didática. Não se presume autorização para ignorar uma orientação adicional do professor que exija especificamente a loja.

2. Tecnologia escolhida e delimitação do enquadramento

O recorte proposto é **gestão híbrida relacional/documental com SQL e JSON em um mesmo SGBD**, associado à pesquisa de gestão multimodelo. A literatura sobre bancos multimodelo discute a convivência de modelos para atender à variedade de dados [1]. O projeto demonstra um recorte limitado dessa ideia, não uma implementação própria de SGBD multimodelo nem uma comparação com bancos de grafos ou vetoriais.

O grupo informou que não havia tecnologia previamente definida/aprovada. O recorte foi escolhido por já existir no código e poder ser demonstrado sem inventar funcionalidades. **MariaDB e JSON não são tecnologias recém-lançadas.** A RFC utilizada é de 2017 e a pesquisa citada é de 2019. A documentação cobre a pesquisa e a demonstração dessa abordagem; a aceitação como “tecnologia emergente” no sentido da disciplina permanece dependente do professor. Se ele exigir uma tecnologia diferente, é necessário alterar o estudo e executar outro experimento, não apenas mudar o título.

3. Fundamentação bibliográfica e técnica

3.1 Variedade de dados e composição de modelos

Lu e Holubová [1] apresentam a gestão multimodelo como uma linha de investigação sobre diferentes representações em uma plataforma de dados. Este estudo usa essa discussão como motivação para combinar um núcleo de vínculos relacionais com alguns documentos, sem tratar todos os tipos de dado como intercambiáveis.

3.2 Documentos e validação

JSON é um formato textual para intercâmbio de dados, conforme a RFC 8259 [2]. No caso estudado, objetos descrevem atributos e arrays representam listas. Isso permite guardar informações adicionais sem acrescentar imediatamente uma coluna à tabela; não elimina a necessidade de definir estrutura, significado e políticas de validação.

O MariaDB documenta seu alias `JSON` como `LONGTEXT`, com validação por `JSON_VALID` [3,4]. A implementação do portal usa explicitamente texto e `CHECK`. Validar sintaxe não valida a regra de negócio: por exemplo, `{"vagas":"muitas"}` pode ser JSON sintaticamente correto e continuar inadequado ao cadastro. A API valida campos, tipos e agenda.

3.3 Índices e portabilidade

Colunas geradas podem materializar ou derivar valores e suportar índices no MariaDB [5]. O portal usa uma coluna gerada para a unicidade de candidaturas ativas; isso não significa que seus atributos JSON já tenham índices derivados. Tal alternativa não foi implementada como parte deste estudo.

PostgreSQL distingue `json` textual e `jsonb` com outra representação e recursos de indexação [6]. Essa diferença serve como alerta de portabilidade: os resultados deste experimento em MariaDB não são resultados de `jsonb` ou de outro SGBD.

As referências completas estão em REFERENCIAS.md. As afirmações sobre o funcionamento do portal são verificadas no schema, migrações e API, não deduzidas apenas das fontes externas.

4. Domínio e aplicação prática

O portal possui três perfis autenticados: administrador geral, responsável por ONG e voluntário. A associação de responsáveis e ONGs é N:N, por `usuarios_ongs`. A ONG organiza projetos; cada projeto oferece oportunidades. Uma candidatura referencia um usuário e uma oportunidade. Valores privados e públicos são separados e não são misturados como se fossem a mesma operação.

O DER atual identifica as tabelas, chaves e cardinalidades. A fonte física combina o schema inicial e sete migrações. O sistema real utiliza:

Núcleo relacional	Informação documental
Projeto, ONG e segmento por FK	Detalhes variáveis da oportunidade em `dados`
Usuário, oportunidade e status da candidatura	Dados da inscrição em `candidaturas.dados`
Conta, e-mail único e perfil	Informações complementares em `dados_voluntario`
Autor, recurso e ação de auditoria	Metadados permitidos em `detalhes`

Valores financeiros permanecem em `DECIMAL(12,2)`, e não em campos JSON arbitrários. Os arquivos de documentos ficam em armazenamento privado; o banco guarda metadados e vínculos. Preferências locais não substituem os cadastros compartilhados.

Exemplo real de documento

```
{
  "titulo": "Apoio nas oficinas",
  "modalidade": "Presencial",
  "local": "Sede fictícia",
  "vagas": 5,
  "diasAtuacao": ["SEG", "QUA"],
  "horarioInicio": "09:00",
  "horarioFim": "11:00",
  "ativa": true
}
```

O documento não contém a única referência do projeto: `oportunidades.projeto_id` é uma FK real. A validação de capacidade/agenda ocorre em transações nas operações relevantes. `JSON_VALID` não deve receber crédito por garantias que são implementadas no PHP ou pelos índices.

5. Comparação com abordagem tradicional

O comparador relacional possui uma linha de oportunidade com atributos estáveis e uma tabela associativa `dias`, com uma linha para cada dia. O comparador documental armazena uma linha por oportunidade e um array `diasAtuacao`. Ambos representam os mesmos 2.000 registros sintéticos e retornam 1.000 oportunidades de segunda-feira.

Critério	Colunas e associação relacional	Documento JSON com IDs relacionais
Lista de dias	Linhas na associação; índice por dia	Array no documento; extração durante a consulta testada
Novo atributo no exemplo	Acrescentar coluna na tabela do experimento	Acrescentar chave no documento
Obrigatoriedade e tipos	Podem ser expressos nas colunas/restrições	Exigem validação além da sintaxe do JSON
Vínculos de negócio	FKs em colunas	FKs mantidas fora dos documentos no portal
Consulta frequente por um atributo	Índice explícito e plano previsível	Pode exigir normalização ou extração indexada
Melhor utilização neste caso	Identidade, estados, valores, relações e consultas frequentes	Complementos variáveis e listas de baixo volume

Essa comparação não implica que todo modelo relacional exija sempre `ALTER TABLE` para extensibilidade: ele também admite tabelas auxiliares ou outros desenhos. O experimento compara dois desenhos explícitos, não todas as alternativas possíveis.

6. Método experimental e resultados

O script `scripts/academic-json-comparison.php` cria apenas tabelas temporárias. Não altera cadastros do portal nem o schema permanente. São usados dados determinísticos fictícios, 20 repetições aquecidas por consulta, mesmo SGBD/conexão e verificação da equivalência dos resultados. O código testa também rejeição de JSON inválido e extensão de um atributo nas duas abordagens.

O relatório experimental apresenta comandos, planos e valores efetivamente medidos. O resultado bruto fica em `resultados-experimento.json`. Não se apresentam números de produção ou uma medição em PostgreSQL.

O teste inicial mostrou retorno equivalente e mediana inferior para o desenho relacional indexado. No exemplo JSON, o novo atributo foi armazenado sem alterar a tabela, mas a busca na lista examinou os documentos. Assim, **a vantagem sustentada é de flexibilidade no exemplo, não de velocidade**. Essa conclusão deve ser mantida mesmo que o desempenho varie entre execuções.

7. Discussão crítica e ameaças à validade

A amostra é pequena, local, sintética e sem usuários concorrentes. A consulta relacional usa índice de dia; o comparador JSON não usa coluna gerada indexada. A diferença de planos faz parte da comparação, mas impede atribuir todo o resultado ao formato de dados. A ordem fixa das consultas, o cache aquecido, o hardware e a versão também limitam a generalização.

Não foram medidos escrita concorrente, custo de backup, tamanho em disco, replicação, JSON indexado ou resultados em outros SGBDs. As tabelas temporárias do microexperimento não possuem FKs entre si; as FKs da aplicação são evidenciadas no DER e nas migrações, e não atribuídas a esse microteste.

No sistema atual, dias e horários são lidos para verificar conflito de agenda. Caso consultas por esses campos cresçam, normalizar a agenda pode ser melhor do que ampliar documentos. A auditoria não preserva cada versão de cada campo, e relatórios por período consideram a situação atual dos registros. Esses limites não devem ser omitidos na demonstração.

8. Conclusão

O ONG Connect mostra que documentos JSON podem coexistir com relações explícitas e transações. O desenho preserva as garantias relacionais onde elas são essenciais e permite representar atributos complementares sem multiplicar imediatamente as colunas. O estudo também mostra o custo dessa flexibilidade: validação semântica adicional, consultas que precisam interpretar documentos e cuidado com indexação e portabilidade.

Para este domínio, a escolha recomendada é seletiva: manter identidades, recursos financeiros, permissões e vínculos em estruturas relacionais e avaliar JSON apenas onde a variedade justifica o custo. A demonstração, o DER e o experimento atendem aos componentes técnicos documentados; o grupo deve confirmar o enquadramento temático com o professor, completar turma/instituição e realizar a apresentação/envio conforme o enunciado.

2. Modelo de dados e relacionamentos

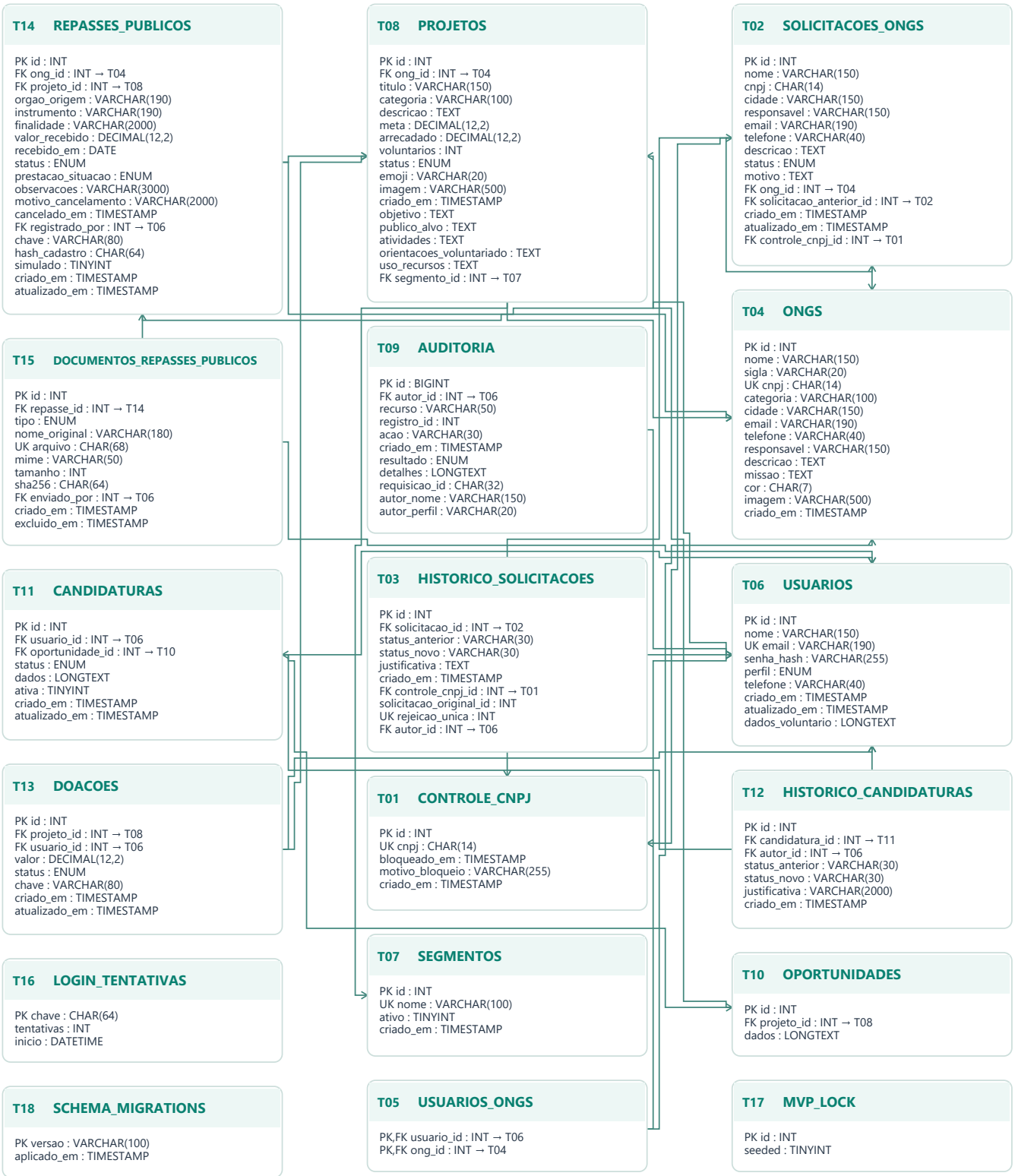
Revisado em **26/10/2026**. A referência física é `database/schema.sql` somado às migrações `001_shared.sql` a `007_segments.sql`, na pasta externa. Não execute apenas o schema inicial para representar o estado final.

Modelo conceitual e cardinalidades

Uma ONG possui projetos e responsáveis vinculados. Cada projeto seleciona um segmento e oferece oportunidades. Usuários voluntários se candidatam a oportunidades; cada candidatura possui decisões históricas. Doações privadas se relacionam a usuário e projeto. Repasses públicos pertencem a uma ONG e, opcionalmente, a um projeto da mesma ONG; podem ter documentos privados. Solicitações de ONG e seu histórico se vinculam ao controle permanente do CNPJ. A auditoria registra ações e pode manter o nome do autor mesmo após exclusão da conta.

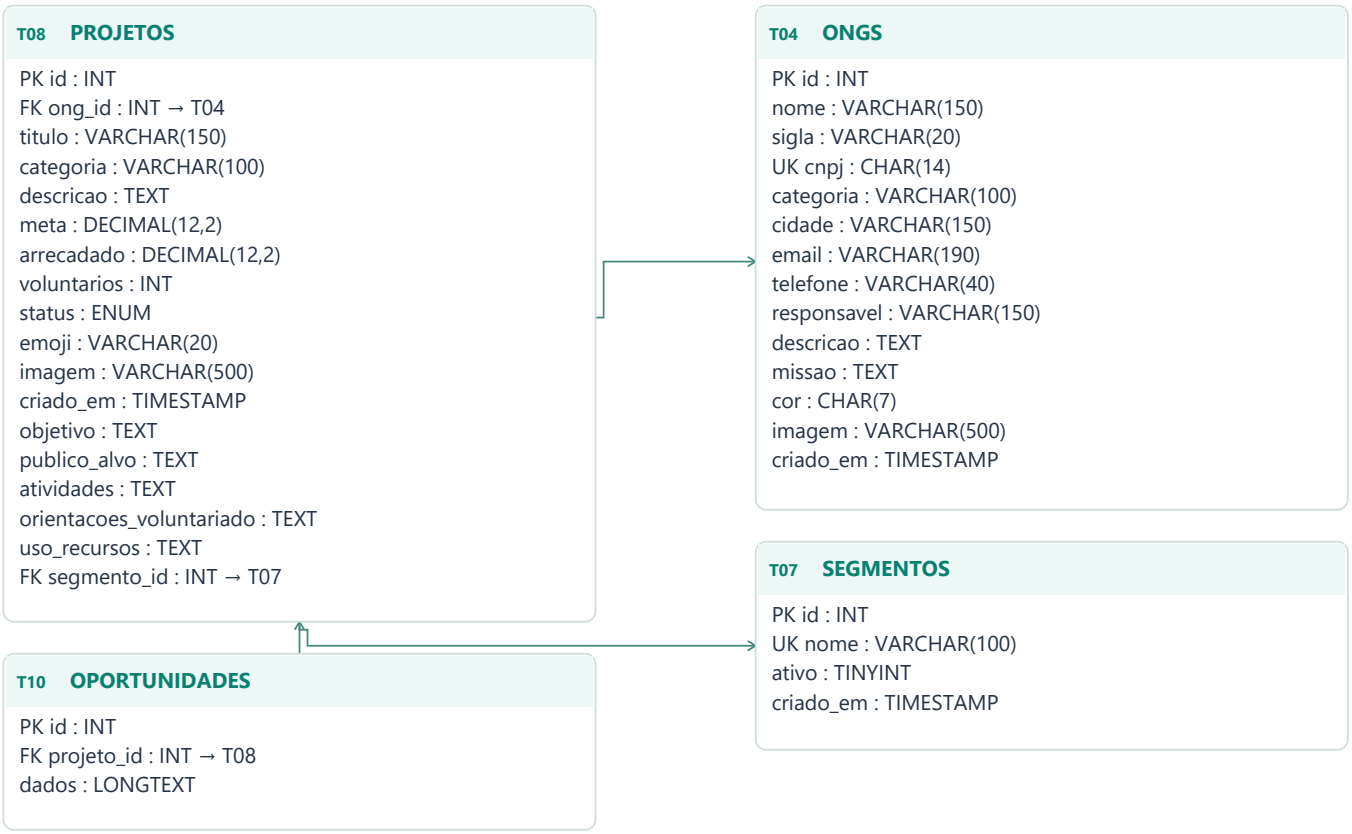
Visão geral das 18 tabelas

Todos os campos físicos do schema e das migrações. Setas apontam para a tabela referenciada; páginas seguintes apresentam os quadros ampliados.



Catálogo e classificação

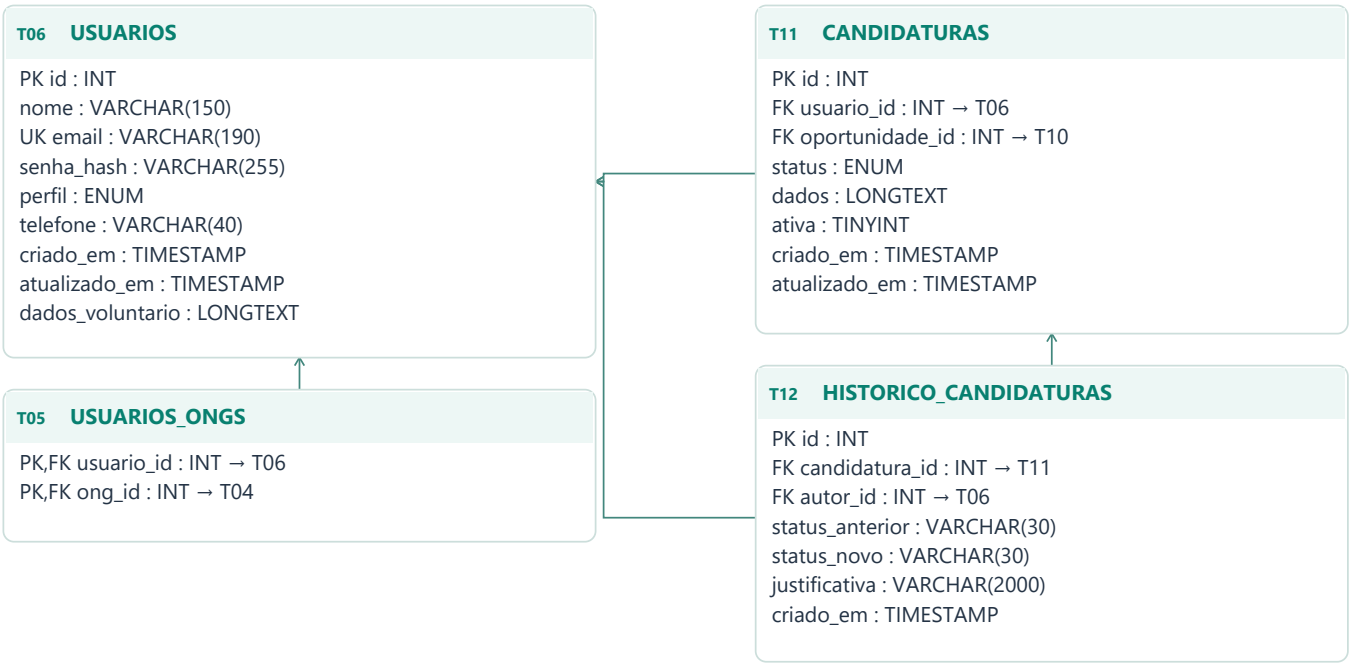
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: ONGS → PROJETOS: 1 : 0..N; SEGMENTOS → PROJETOS: 0..1 : 0..N; PROJETOS → OPORTUNIDADES: 1 : 0..N.

Contas e candidaturas

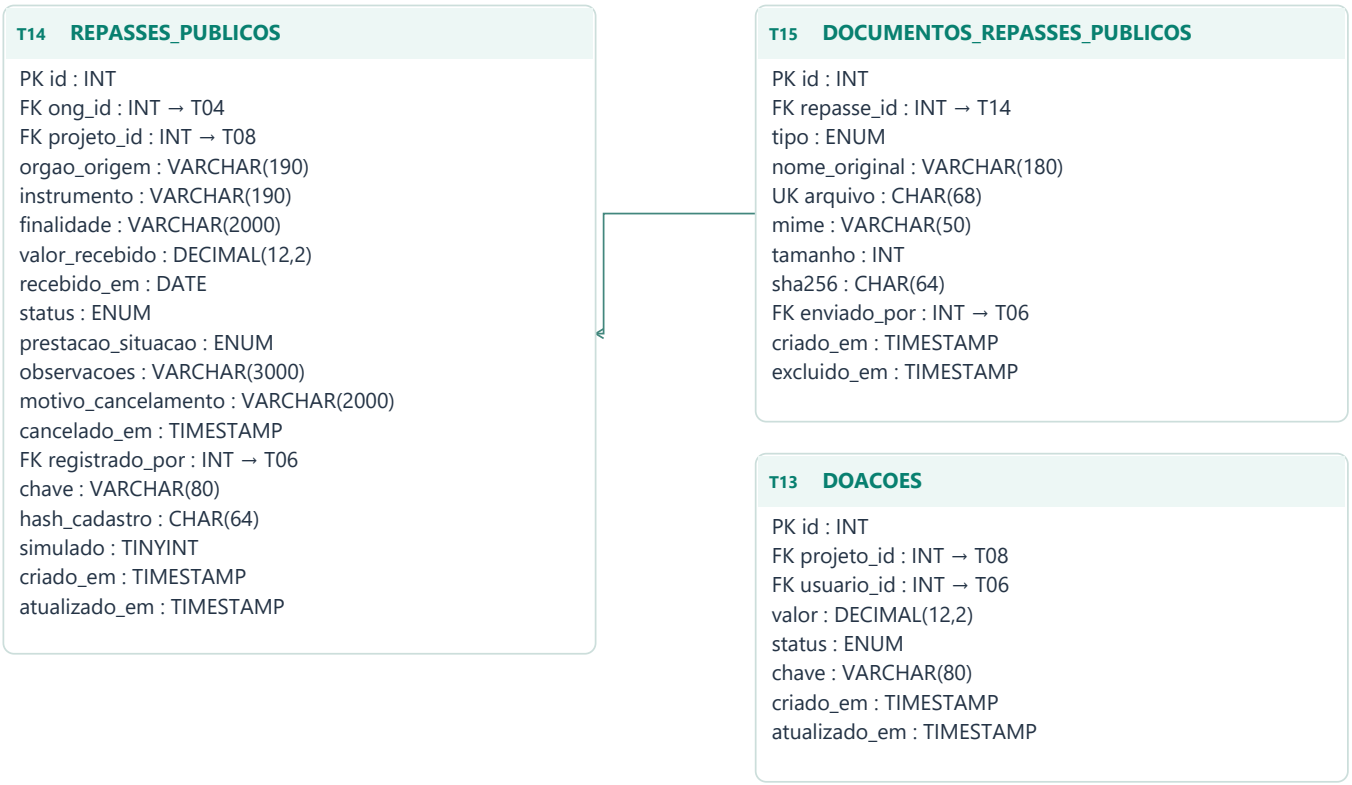
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: USUARIOS → USUARIOS_ONGS: 1 : 0..N; USUARIOS → CANDIDATURAS: 1 : 0..N; CANDIDATURAS → HISTORICO_CANDIDATURAS: 1 : 0..N; USUARIOS → HISTORICO_CANDIDATURAS: 1 : 0..N.

Doações, repasses e documentos

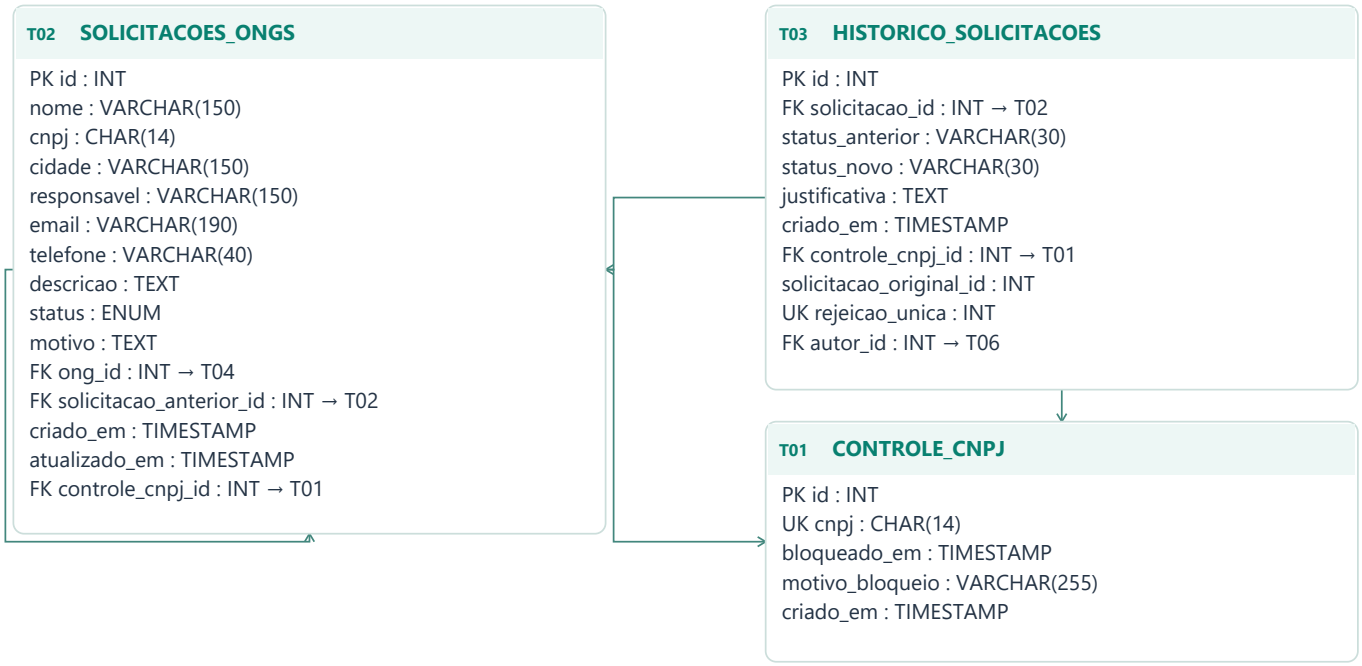
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: REPASSES_PUBLICOS → DOCUMENTOS_REPASSES_PUBLICOS: 1 : 0..N.

Solicitações e controle de CNPJ

Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: CONTROLE_CNPJ → SOLICITACOES_ONGS: 1 : 0..N; CONTROLE_CNPJ → HISTORICO_SOLICITACOES: 1 : 0..N; SOLICITACOES_ONGS → SOLICITACOES_ONGS: 0..1 : 0..N; SOLICITACOES_ONGS → HISTORICO_SOLICITACOES: 0..1 : 0..N.

Auditoria e tabelas técnicas

Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.

T09 AUDITORIA

PK id : BIGINT
FK autor_id : INT → T06
recurso : VARCHAR(50)
registro_id : INT
acao : VARCHAR(30)
criado_em : TIMESTAMP
resultado : ENUM
detalhes : LONGTEXT
requisicao_id : CHAR(32)
autor_nome : VARCHAR(150)
autor_perfil : VARCHAR(20)

T16 LOGIN_TENTATIVAS

PK chave : CHAR(64)
tentativas : INT
inicio : DATETIME

T17 MVP_LOCK

PK id : INT
seeded : TINYINT

T18 SCHEMA_MIGRATIONS

PK versao : VARCHAR(100)
aplicado_em : TIMESTAMP

O diagrama representa os principais atributos e todas as tabelas, sem repetir cada campo de apresentação e timestamp. PK é chave primária, FK é estrangeira e UK indica unicidade. As três tabelas técnicas isoladas não têm relacionamentos de negócio. As cardinalidades opcionais representam a nulabilidade física; não eliminam exigências adicionais da API.

Dicionário e regras verificáveis

Tabela	Responsabilidade e restrições
`ongs`	Identificação e contato institucional; CNPJ único quando preenchido
`usuarios`	Conta, hash da senha, perfil e dados do voluntário; e-mail único
`usuarios_ongs`	Associação N:N de contas responsáveis e ONGs; PK composta evita vínculo repetido
`segmentos`	Vocabulário controlado; nome único, ativação/desativação sem apagar projetos
`projetos`	FK para ONG/segmento, descrições e meta; totais antigos `arrecadado`/`voluntarios` são legados, não a origem dos indicadores atuais
`oportunidades`	FK para projeto e JSON de título, vagas, modalidade, local, dias, horários e abertura
`candidaturas`	FK para usuário/oportunidade; índice único `(usuario_id,oportunidade_id,ativa)` impede duplicidade ativa e permite histórico inativo
`historico_candidaturas`	Transições com autor, justificativa e data; não substitui toda a fotografia de cada campo
`doacoes`	Valor positivo `DECIMAL(12,2)` e chave única por usuário; simulação idempotente
`repasses_publicos`	Recebimento positivo, origem, instrumento, prestação de contas e cancelamento; chave composta `(projeto_id,ong_id)` impede projeto de outra ONG
`documentos_repasses_publicos`	Metadados do arquivo, autor, integridade SHA-256 e exclusão lógica; conteúdo não fica no banco
`controle_cnpj`	Chave normalizada persistente, bloqueio após três rejeições distintas
`solicitacoes_ongs`	Pedido, situação, ligação com controle e eventual ONG publicada
`historico_solicitacoes`	Decisões preservadas após exclusão; rejeição contabilizada uma vez por solicitação
`auditoria`	Autor, alvo, ação, resultado e correlação; permite ações públicas sem autor autenticado
`login_tentativas`	Contador e janela temporal para limitar tentativas
`mvp_lock`	Serialização de determinadas gravações e controle da carga inicial
`schema_migrations`	Versões aplicadas pelo migrador

Integridade no banco e na API

FKs garantem existência dos vínculos. `DECIMAL` mantém representação decimal dos valores armazenados. Índices únicos protegem CNPJ, e-mail e reenvios. Transações e bloqueios impedem exceder a última vaga e preservam a sequência das decisões. A API valida perfis, dias/horários, capacidade, datas e escopo da ONG; essas regras não são garantidas apenas pelo DER.

A categoria textual de `projetos` é mantida por compatibilidade e atualizada junto ao segmento. O ID é a referência controlada. Uma alteração SQL direta que ignore a API pode desalinhar a cópia do nome; não se declara normalização completa de todos os campos. `dados` e `dados_voluntario` também são uma escolha de desnormalização parcial, analisada no trabalho.

Relacional + JSON efetivamente utilizado

Os documentos são guardados em `LONGTEXT` com `CHECK(JSON_VALID(...))`, e não em `jsonb`. Exemplos: `oportunidades.dados`, `candidaturas.dados`, `usuarios.dados_voluntario` e `auditoria.detalhes`. IDs de negócio permanecem em colunas relacionais. A validação de JSON não obriga um documento a ter um número positivo de vagas nem valida relações; essas verificações estão no PHP.

No modelo tradicional alternativo do experimento, dias ficam em uma tabela associativa por oportunidade. No modelo efetivo, a lista é parte do documento. Consulte experimento para a comparação reproduzível; ele não altera o schema do portal.

Exclusões, documentos e histórico

Solicitações removidas podem deixar seu histórico ligado ao controle permanente de CNPJ, com FK da solicitação nula. Doações/repasses cancelados permanecem registrados; cancelamento não é exclusão. Documentos usam `excluido_em`; os backups precisam abranger banco e armazenamento privado. A auditoria não é um log inviolável de todas as versões e não tem uma FK para cada possível `registro_id`.

Evolução física

Migração	Alteração
`001_shared`	Contas, vínculos, candidaturas, decisões, doações, auditoria e tentativas de login
`002_legacy_schedule`	Compatibilidade da agenda legada
`003_cnpj_audit`	Controle permanente de CNPJ, preservação de histórico e auditoria ampliada
`004_project_details`	Objetivo, público, atividades e orientações de projetos
`005_public_transfers`	Repasses públicos e metadados dos documentos privados
`006_volunteer_profile`	Documento JSON de perfil do voluntário
`007_segments`	Segmentos controlados e vínculo aos projetos

As preferências do navegador não integram o DER. SQL, migrações e testes são a evidência executável das restrições.

3. Experimento e resultados

Executado localmente em **26/10/2026**; data e fuso da execução constam no resultado bruto. Script: `scripts/academic-json-comparison.php`, na pasta externa.

Perguntas verificadas

1. As duas representações retornam o mesmo conjunto esperado de oportunidades por dia?

2. Um atributo adicional pode ser colocado no JSON sem acrescentar coluna?

3. O banco rejeita sintaxe JSON inválida quando existe `CHECK(JSON_VALID(...))`?

4. Como se comportam os planos e tempos das duas consultas específicas?

Ambiente e dados

• PHP **8.5.12**, MariaDB **11.4.5-MariaDB**, conexão local PDO; sem acesso ao banco remoto.

• 2.000 oportunidades fictícias em cada desenho, IDs de 1 a 2.000.

• IDs ímpares têm SEG/QUA e pares têm TER/QUI, resultando em 4.000 linhas de dias na alternativa normalizada.

• Mesmos títulos, capacidades e distribuição; cada consulta por SEG encontra 1.000 oportunidades.

• Uma consulta inicial de aquecimento, seguida de 20 repetições medidas por `hrtime`.

• Mediana das posições centrais ordenadas e p95 pelo método da posição 19 em 20 (nearest rank).

• Tabelas `TEMPORARY` com InnoDB e prefixo `academic_`, restritas à conexão. Não modificam schema nem registros permanentes; desaparecem ao fechar a conexão. O código não usa `DROP DATABASE` nem cargas no portal.

Desenhos comparados

```
-- Relacional: colunas e associação para os dias.
CREATE TEMPORARY TABLE academic_rel (
  id INT PRIMARY KEY, titulo VARCHAR(100), vagas INT NOT NULL
) ENGINE=InnoDB;
CREATE TEMPORARY TABLE academic_dias (
  oportunidade_id INT NOT NULL, dia CHAR(3) NOT NULL,
  PRIMARY KEY(oportunidade_id,dia), INDEX por_dia(dia,oportunidade_id)
) ENGINE=InnoDB;

-- Documental dentro do mesmo SGBD.
CREATE TEMPORARY TABLE academic_json (
  id INT PRIMARY KEY,
  dados LONGTEXT NOT NULL CHECK(JSON_VALID(dados))
) ENGINE=InnoDB;
```

As tabelas temporárias não implementam FKs entre si. A demonstração das FKs pertence ao modelo real do portal, que contém a relação oportunidade–projeto e os demais vínculos. Não se atribui ao microexperimento uma garantia que ele não testa.

```
SELECT COUNT(*)
FROM academic_rel o
JOIN academic_dias d ON d.oportunidade_id=o.id
WHERE d.dia='SEG';

SELECT COUNT(*)
FROM academic_json
WHERE JSON_CONTAINS(dados,'"SEG"','$.diasAtuacao');
```

Resultados da execução registrada

Medida	Relacional indexado	JSON sem índice derivado
Registros retornados na contagem	1000	1000
Mínimo por consulta	2.022 ms	3.205 ms
Mediana por consulta	2.138 ms	3.375 ms
p95 por consulta	2.68 ms	4.236 ms
Repetições medidas	20	20

O `EXPLAIN` foi gravado no JSON dos resultados. Na execução registrada, a alternativa relacional utilizou o índice `por_dia` e lookup pela PK; a alternativa documental realizou leitura integral para avaliar o array. A medição não é evidência de que todo uso de JSON é lento: é o resultado destes desenhos e desta consulta.

O teste também confirmou:

• JSON malformado rejeitado pelo banco: **sim**.

• Chave `acessibilidade` acrescentada via `JSON_SET`, sem `ALTER TABLE`: **sim**.

• Coluna `acessibilidade` acrescentada ao desenho relacional com `ALTER TABLE`, seguida de atualização: executado no ambiente temporário.

A facilidade de adicionar a chave foi observada funcionalmente, não como um benchmark de custo de migração. Outros desenhos relacionais também poderiam tratar extensibilidade de outras formas.

Reproduzir

Com PHP disponível no PATH e extensões carregadas:

```
php scripts/academic-json-comparison.php
```

No ambiente portátil preparado, execute na pasta externa:

```
& ".runtime/php/php.exe" -n -d extension_dir=.runtime/php/ext -d extension=pdo_mysql -d
extension=mbstring scripts/academic-json-comparison.php
```

A conexão usa a configuração privada já existente ou as variáveis de ambiente suportadas pelo projeto. O usuário SQL precisa de permissão para criar tabelas temporárias. Não é preciso substituir o banco da aplicação. O script aborta com erro se a equivalência, a validação ou a extensão do atributo falhar.

Interpretação e limites

O benefício observado do documento é a flexibilidade da representação complementar; a consulta relacional indexada foi mais rápida nesta execução. Não se generalizam os números para PostgreSQL, MySQL, produção, outro volume ou JSON com coluna gerada indexada.

A avaliação é local, aquecida, com uma distribuição artificial, 20 repetições e ordem fixa dos desenhos. Não mede rede, múltiplos usuários, escrita concorrente, custo em disco ou migração real de milhares de linhas permanentes. O índice na associação e a ausência de índice derivado no documento são diferenças explícitas. Cache e ruído do computador afetam as medições.

O resultado não prova que o portal deve ser refeito nem que foi implementado um novo SGBD. Ele fornece uma comparação pequena e reproduzível para discutir escolhas de modelagem.

4. Requisitos e permissões

Revisado em 26/10/2026. Estes requisitos descrevem a aplicação; os requisitos da disciplina estão na matriz acadêmica.

ID	Requisito
RF01	Consultar ONGs, projetos, vagas e transparência pública
RF02	Registrar e revisar solicitações de ONG, preservando histórico e controle de CNPJ
RF03	Bloquear novas solicitações após três rejeições distintas do mesmo CNPJ
RF04	Autenticar contas e restringir operações por perfil e vínculo da ONG
RF05	Cadastrar projetos completos e selecionar segmentos gerenciados pelo administrador
RF06	Gerenciar oportunidades, capacidade, agenda e abertura de inscrições
RF07	Registrar candidaturas compartilhadas com duplicidade e conflito de agenda controlados
RF08	Analisar, cancelar e promover candidaturas manualmente, com histórico
RF09	Registrar doações simuladas com reenvio idempotente e cancelamento
RF10	Separar repasses públicos, documentos privados e prestação de contas
RF11	Consolidar relatórios por tipo, período e ONG, exportando CSV
RF12	Consultar auditoria exclusivamente como administrador geral

Restrições: chaves estrangeiras, transações, DECIMAL para valores, CSRF, senha com hash, validação no backend e testes de isolamento. O portal usa dados fictícios; não processa pagamentos nem representa aprovação de contas por órgão público. A auditoria não guarda todas as versões anteriores de cada campo. A documentação distingue recursos efetivos, experimentos e propostas futuras.

5. Casos de uso

Revisado em 26/10/2026. As permissões são reais e verificadas pela API.

Ator	Ações
Visitante	Consultar catálogo/transparência e solicitar cadastro de ONG
Voluntário autenticado	Editar seu perfil, candidatar-se, acompanhar/cancelar suas candidaturas e simular apoio
Responsável por ONG	Gerenciar projetos, vagas, candidaturas e repasses das ONGs vinculadas; consultar relatórios desse escopo
Administrador geral	Gerenciar ONGs, solicitações e segmentos; consultar todo o escopo administrativo e auditoria

Ao escolher uma vaga sem autenticação, a pessoa escolhe entrar ou cadastrar-se. Após o acesso, retorna à oportunidade; a candidatura exige envio explícito. Na lotação, pode entrar em espera para análise manual. Aprovação não pode ultrapassar a capacidade nem gerar conflito de agenda.

Relatórios consultam resultados; Repasses opera recebimentos e documentos; Auditoria investiga quem realizou ações. Nenhuma dessas consultas substitui um pagamento real ou uma análise jurídica de prestação de contas.

6. Arquitetura

Revisado em 26/10/2026. Aplicação multipágina; não é SPA. Frontend sem framework, API PHP/PDO, MariaDB e arquivos privados.

`config/`, `database/`, `scripts/` e `storage/` ficam fora da raiz pública `ong-connect/`. A API valida entradas, CSRF e escopo da ONG. Operações críticas usam transações e bloqueios para capacidade, filas, idempotência e controle de CNPJ.

Componente	Responsabilidade
`api/index.php` e `bootstrap.php`	Roteamento, catálogo, validações e PDO
`api/security.php`	Sessão, perfis, vínculos e auditoria
`api/participation.php`	Candidaturas e doações
`api/public-transfers.php`	Repasses e documentos privados
`api/reports.php`	Consultas consolidadas, sem gravar relatórios
`api/segments.php`	Segmentos controlados
`assets/js/app.js`, `navigation.js`, `account.js`	Base pública, navegação e conta
`assets/js/reports-admin.js`, `repasses-publicos.js`, `auditoria.js`	Telas com finalidades separadas

`loadData()` consulta o catálogo e, na versão atual, também pode carregar registros compartilhados para usuários autenticados. A varredura de desempenho identificou dependências excessivas; as otimizações ainda não foram aplicadas. Não se deve descrever cache de dados privados ou navegação persistente como funcionalidades prontas.

7. Instalação e operação

Referência vigente em 26/10/2026. Leia também README e backend.

Instalação limpa

Requisitos: PHP/PDO com `pdo_mysql`, `mbstring`, `fileinfo`; MariaDB com InnoDB e utf8mb4; servidor que execute PHP. Node.js e Chrome são ferramentas de testes, não dependências do usuário do portal.

Na pasta externa, configure `config/database.php` conforme o exemplo, crie o banco e execute:

```
php scripts/install.php
$env:ONG_INITIAL_PASSWORD = Read-Host 'Senha inicial fictícia, com ao menos 12 caracteres'
php scripts/create-user.php admin@example.com 'Administrador acadêmico' admin
Remove-Item Env:ONG_INITIAL_PASSWORD
php -S 127.0.0.1:8080 -t ong-connect
```

Use uma senha de demonstração exclusiva. O comando não deve receber senhas reais copiadas de documentação. No ambiente portátil preparado, `scripts/start-local.ps1` inicia PHP e MariaDB com as extensões necessárias; os binários em `.runtime/` não são distribuídos como parte do código.

Atualização e backup

Faça backup do banco, arquivos públicos, configuração privada e documentos antes de atualizar. Execute `php scripts/migrate.php`; a versão aplicada é registrada em `schema_migrations`. Não importe o banco local sobre o servidor para sincronizar alterações de interface. Credenciais e uploads são preservados separadamente.

`database/schema.sql` é a base inicial. As sete migrações acrescentam contas/vínculos, agenda, CNPJ/auditoria, detalhes de projetos, repasses/documentos, perfil do voluntário e segmentos. A instalação básica utiliza os JSON iniciais; exemplos adicionais e dados de testes não equivalem ao banco remoto.

Persistência e segurança

Contas e dados de negócio ficam no MariaDB, inclusive candidaturas e doações. Preferências de foto/filtros podem ficar no navegador. Documentos não são BLOBs: a tabela guarda seus metadados, e o arquivo fica em diretório privado. O download passa por autorização da API.

Login é real, por cookie de sessão. A API verifica perfis, vínculos e CSRF. Não existe conta administrativa padrão com senha pública. `create-user.php` permite criar responsável por ONG com o argumento adicional do ID da ONG; publicação aprovada também pode provisionar esse acesso.

Consultas principais

Recurso da API	Finalidade
`catalogo`, `total_repasses_publicos`	Consulta pública; o segundo retorna total global recebido
`sessao`, `login`, `cadastro`, `perfil`, `logout`	Autenticação e conta
`solicitacoes`, `publicar`, `rejeitar`, `ongs`, `projetos`, `oportunidades`	Gestão com permissões
`segmentos`	Gestão exclusiva do administrador; lista de seleção no catálogo
`candidaturas`, `doacoes`	Participações e contribuições simuladas
`repasses_publicos`, `documentos_repasses_publicos`, `contexto_repasses_publicos`	Operação privada de repasses
`relatorios`	Financeiro/projetos/voluntariado por período e ONG autorizada
`auditoria`	Leitura exclusiva do administrador geral

Relatórios usam `inicio`, `fim`, `tipo` e `ongId`. Os filtros são validados no servidor. Cancelados não integram o total recebido. API privada não deve ser armazenada em cache público. Consulte testes antes de uma publicação.

8. Referências

Consultadas em **26/10/2026**. Fontes primárias; paráfrases próprias, sem reprodução integral dos textos. Os números correspondem às citações do relatório.

1. **LU, Jiaheng; HOLUBOVÁ, Irena.** *Multi-model Databases: A New Journey to Handle the Variety of Data*. ACM Computing Surveys, v. 52, n. 3, artigo 55, 2019. DOI: 10.1145/3323214. Registro e manuscrito dos autores na Universidade de Helsinki. Fundamenta o contexto de gestão de dados variados; não certifica a implementação do ONG Connect.

2. **BRAY, T. (ed.); IETF.** *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, dezembro de 2017. Texto oficial. Referência para o formato JSON; sua data evidencia que JSON não é uma criação recente.

3. **MARIADB.** *JSON Data Type*. Documentação oficial. Página. Referência do armazenamento e da diferença em relação a outras implementações.

4. **MARIADB.** *JSON_VALID*. Documentação oficial. Página. Referência de validação sintática.

5. **MARIADB.** *Generated Columns*. Documentação oficial. Página. Referência para colunas geradas e alternativas de indexação.

6. **POSTGRESQL GLOBAL DEVELOPMENT GROUP.** *JSON Types*, documentação PostgreSQL 16, seção 8.14. Página da versão consultada. Utilizada somente para discutir diferenças de implementação e portabilidade; PostgreSQL não foi executado no experimento.

Fontes primárias do próprio projeto

• Enunciado fornecido: **Projeto da disciplina — Modelagem de Banco de Dados — Terças (1).pdf**, duas páginas. Define os componentes da atividade, a data, o limite de integrantes e o exemplo de loja. O arquivo permanece no local disponibilizado pelo usuário; não é redistribuído nem convertido em instrução para envio de e-mail.

• `database/schema.sql` e `database/migrations/001...007`: evidência do modelo físico.

• `ong-connect/api/`: evidência das regras efetivamente verificadas no backend.

• `scripts/academic-json-comparison.php` e resultado bruto: evidência do experimento.

• `scripts/test-shared.cjs` e suítes chamadas: evidência dos testes funcionais descritos, separados do experimento de modelagem.

O formato acima é uma referência identificável e verificável; não se declara conformidade formal com um padrão bibliográfico que não foi exigido no PDF. Datas e critérios de acesso devem ser revistos se a equipe atualizar o estudo.

9. Matriz de conformidade

Revisão de **26/10/2026**. Esta matriz distingue material produzido de aceitação pelo professor. Não constitui nota, homologação ou confirmação de entrega.

Solicitação do enunciado	Evidência preparada	Situação
Selecionar tecnologia emergente	Recorte proposto: modelagem híbrida relacional/documental com JSON, seções 2 e 3 do relatório	Proposta documentada; aceitação do enquadramento ainda não confirmada
Estudar fundamentos e características	Revisão bibliográfica/técnica e fontes primárias identificadas	Documentado
Identificar limitações	Discussão crítica, ameaças à validade e limites da aplicação	Documentado
Pequeno exemplo de aplicação	Portal funcional e exemplo de oportunidades de voluntariado	Implementado
Diagrama entidade-relacionamento	Modelo atual e cardinalidades em `09-ESTRUTURA-DE-DADOS.md`	Atualizado; renderização visual pode ser conferida no editor antes da apresentação
Comparar com abordagem tradicional	Tabela comparativa, código dos dois desenhos e experimento local	Executado e documentado, dentro do escopo informado
Mostrar benefícios e situações de uso	Conclusão seletiva: flexibilidade versus validação/indexação	Documentado sem alegar superioridade universal
Apresentar resultados	Resultados brutos, interpretação e roteiro de demonstração	Material preparado; apresentação ainda depende do grupo
Grupo de até sete integrantes	Sete nomes e RGMs informados, registrados no relatório e README	Identificação dos integrantes preenchida; limite atendido
Apresentação em 27/10/2026 e envio por e-mail	Checklist do roteiro; endereço consta no PDF original	Não executado; nenhuma mensagem foi enviada

Interpretação do domínio

A página 2 é introduzida como exemplo de domínio de loja virtual. Não há no material lido uma regra explícita que obrigue usar esse domínio. O estudo usa ONGs; qualquer orientação adicional do professor deve ser respeitada.

Pendências objetivas antes de entregar

1. Confirmar se o professor aceita este recorte tecnológico. Não resolver esse ponto apenas chamando JSON de "novo".

2. Conferir a identificação: Modelagem de Bancos de Dados — terça-feira, noite; Cruzeiro do Sul — Campus Paulista; os sete integrantes e RGMs já foram registrados.

3. Abrir o DER em um visualizador Mermaid ou exportá-lo para imagem/PDF, caso o formato escolhido para apresentação exija isso.

4. Ensaiar os comandos no computador da apresentação, usando somente dados fictícios.

5. Preparar os arquivos no formato de entrega escolhido e enviar ao destinatário do enunciado pelo próprio grupo.

O PDF não especifica número de slides, quantidade de páginas, padrão ABNT ou ferramenta obrigatória para o DER. Esses itens não são apresentados como exigências inventadas.

10. Roteiro de apresentação

Material revisado em 26/10/2026. A duração abaixo é apenas uma sugestão de organização; o PDF não impõe tempo ou quantidade de slides.

Identificação

Título: **Modelagem híbrida relacional e documental com JSON: ONG Connect**. Os sete integrantes e RGMs estão identificados no relatório acadêmico. Turma: Modelagem de Bancos de Dados — terça-feira, noite. Instituição: Cruzeiro do Sul — Campus Paulista. O grupo respeita o limite de sete pessoas do enunciado. Data prevista: 27/10/2026. Confirmar o recorte com o professor antes de declarar o requisito temático atendido.

Sequência sugerida

Etapa	O que apresentar	Evidência
1	Problema de atributos variados nas oportunidades	Cadastro de vaga e pergunta de pesquisa
2	Gestão de dados heterogêneos e limites do recorte	Fundamentação e referências
3	Entidades, cardinalidades e integridade	DER atual, FKs e índices nas migrações
4	Uso concreto de JSON	Coluna `oportunidades.dados` e validação no PHP
5	Abordagem tradicional	Tabela da oportunidade e associação de dias
6	Experimento	Execução do script, igualdade de contagem e planos
7	Resultados, benefícios e custos	Medianas medidas, validação e extensão do atributo
8	Conclusão e limites	Quando normalizar, quando usar documento e o que não foi medido

Demonstração reproduzível

1. Iniciar o ambiente local e entrar com uma conta fictícia autorizada. Não projetar senhas, configurações privadas ou contas reais.

2. Abrir Projetos e uma oportunidade; mostrar o vínculo com ONG/projeto e a lista de dias.

3. Abrir o DER e explicar o relacionamento usuário–candidatura–oportunidade.

4. Executar `scripts/academic-json-comparison.php` pelo PHP com `pdo_mysql`, seguindo EXPERIMENTO.md.

5. Mostrar que ambas as consultas encontram 1.000 registros; interpretar os tempos como microexperimento local.

6. Mostrar a rejeição de JSON inválido e a adição de atributo sem alteração de coluna na alternativa documental.

7. Explicar que isso não garante validação semântica e que o comparador relacional possui índice de dia.

8. Se houver tempo, mostrar a separação entre relatórios, operação de repasses e auditoria, como aplicação da persistência e das permissões.

Se a execução falhar, apresentar os resultados registrados como execução anterior, sem fingir demonstração ao vivo. Não alterar o banco do servidor para ensaiar.

Perguntas que o grupo deve saber responder

• Por que guardar o projeto em FK, em vez de apenas dentro do JSON?

• O que `JSON_VALID` garante e o que continua sendo responsabilidade da API?

• Por que o teste relacional foi mais rápido na execução observada?

• Por que um campo novo sem `ALTER TABLE` não elimina a necessidade de governança do esquema?

• Por que os resultados não se aplicam automaticamente a `jsonb` ou a qualquer banco NoSQL?

• Quais dados continuam no navegador e quais são compartilhados pelo servidor?

Checklist de entrega

• [ ] Confirmar o enquadramento da tecnologia com o professor.

• [x] Identificar os sete integrantes e RGMs e conferir o limite do grupo.

• [x] Identificar turma e instituição: Modelagem de Bancos de Dados — terça-feira, noite; Cruzeiro do Sul — Campus Paulista.

• [ ] Verificar links, legibilidade do DER e resultados do experimento.

• [ ] Separar relatório, referências, diagrama, código e roteiro no formato escolhido.

• [ ] Excluir credenciais, `.runtime/`, backups, dados pessoais e documentos privados do pacote a compartilhar.

• [ ] Apresentar e enviar conforme data e destinatário do PDF original.

Esta revisão não realizou envio de e-mail nem publicação no servidor.

11. Segurança e melhorias para operação online

Revisão documental: **27/10/2026**. O grupo informa que o sistema está online. Esta análise combina inspeção do PDF de entrega e revisão estática da cópia local do código. Não foram realizados testes contra o servidor, acesso a configurações privadas ou verificação da versão publicada. Os achados locais não demonstram, por si, que o servidor está vulnerável.

Escopo e situação das recomendações

Os itens abaixo são **melhorias propostas, ainda não implementadas nem validadas por esta revisão**. A prioridade indica a ordem sugerida de tratamento, considerando impacto potencial e exposição. Não corresponde a uma pontuação CVSS nem a um teste de invasão. Controles eventualmente existentes no proxy, firewall ou hospedagem precisam ser conferidos antes de concluir sobre a exposição real.

Controles já identificados no código

• Consultas com parâmetros preparados no PDO e preparação emulada desativada.

• Hash de senhas com `password_hash` e verificação com `password_verify`.

• Tokens CSRF nas operações autenticadas, controle por perfil e autorização por vínculo à ONG.

• Modo estrito de sessão, cookie HttpOnly/SameSite e regeneração do identificador após login.

• Downloads sujeitos à autenticação e ao escopo da ONG, com verificação de SHA-256, disposição como anexo e proteção contra interpretação do conteúdo.

• Uploads com MIME permitido, nomes aleatórios, limite por arquivo e permissões de armazenamento restritas.

• Mensagens de erro genéricas para falhas internas, sem devolver a exceção técnica ao usuário.

Essas evidências não constituem certificação de segurança nem comprovam a configuração do ambiente publicado.

1. Cadastro público de solicitações de ONG

Prioridade alta — fragilidade observada na aplicação local. Em `ong-connect/api/index.php`, o POST de solicitações é público e fica fora da exigência geral de CSRF. Não foi identificado controle de frequência nessa rota. O processamento de gravações também utiliza bloqueio compartilhado por `mvp_lock`.

O risco principal é automação de cadastros, crescimento indevido de registros e disputa pelo bloqueio de escrita. A ausência de autenticação é compatível com um formulário público; o problema é a falta de controles de abuso na aplicação. A ausência de CSRF também permite considerar submissões induzidas por outros sites, conforme o formato aceito pela rota. Não foi demonstrada exploração no servidor.

Melhoria proposta: limites por IP e janela temporal, proteção contra automação, validação de origem/CSRF conforme o fluxo e redução do alcance do bloqueio. CSRF não substitui controles contra robôs.

Critério de validação: em homologação, excesso de envios deve receber 429 sem criar novos registros; solicitações legítimas devem continuar funcionando. Medir a concorrência para evitar que o formulário público prejudique outras gravações.

2. Ativação e senha inicial de responsáveis por ONG

Prioridade alta — limitação observada no fluxo local. A função `provisionOngAccess`, em `ong-connect/api/security.php`, gera senha aleatória e retorna `senhaInicial` ao administrador. Não foi identificado na API revisada um fluxo de ativação pelo titular ou troca dessa senha. A senha é aleatória e armazenada com hash; a preocupação está na distribuição e no ciclo de vida da credencial.

Melhoria proposta: substituir a entrega de senha por convite com token de uso único, expiração e armazenamento do hash do token. O responsável define a própria senha, por canal verificado. Prever recuperação de acesso e revogação de sessões quando necessário. Não registrar tokens ou senhas em logs.

Critério de validação: convite expirado ou reutilizado deve ser recusado; a API não deve retornar senha inicial ao administrador; o titular deve conseguir ativar e recuperar sua conta de forma controlada.

3. Proteção contra tentativas de login

Prioridade média — limitação observada na aplicação local. Em `security.php`, a chave do contador deriva apenas de `REMOTE_ADDR`, e o registro é apagado após autenticação bem-sucedida. Há limitação por IP, mas ela não cobre adequadamente tentativas distribuídas contra a mesma conta. Usuários de uma rede compartilhada também podem afetar o limite uns dos outros.

Melhoria proposta: combinar limites por conta e IP, atraso progressivo, mensagens genéricas e alertas para padrões suspeitos. Evitar bloqueio permanente que possa ser provocado contra terceiros. Considerar autenticação multifator para administradores.

Critério de validação: testar ataques distribuídos simulados e usuários atrás do mesmo endereço em homologação; um login legítimo não deve apagar toda a proteção contra abuso de outras contas.

4. HTTPS, proxy e cookie de sessão

Prioridade condicionada à hospedagem — exposição não confirmada. O atributo Secure é calculado a partir de `$_SERVER['HTTPS']`. Se um proxy terminar o HTTPS sem informar corretamente o protocolo ao PHP, o cookie poderá ser emitido sem esse atributo.

Melhoria proposta: conferir a topologia da hospedagem, forçar HTTPS e assegurar Secure no ambiente de produção. Quando houver proxy, aceitar informação de protocolo apenas de proxies confiáveis. Avaliar HSTS após validar o HTTPS e seu alcance. Conferir também CSP e proteção contra enquadramento nas páginas HTML; não presumir sua ausência apenas por não estarem na API.

Critério de validação: observar a resposta efetiva de produção e confirmar Secure, HttpOnly e SameSite no cookie, além do redirecionamento HTTP para HTTPS. Não confiar indiscriminadamente em cabeçalhos encaminhados pelo cliente.

5. Uploads e capacidade de armazenamento

Prioridade média — oportunidade de endurecimento. `ong-connect/api/public-transfers.php` verifica tamanho de até 2 MB, MIME PDF/JPEG/PNG, autorização, hash e nomes aleatórios. Não foi identificada nessa implementação cota acumulada por ONG nem análise antimalware. MIME e SHA-256 não demonstram que o conteúdo de um arquivo é seguro.

Melhoria proposta: cotas por organização, limites de frequência, monitoramento de disco e análise dos arquivos conforme o uso. Manter o conteúdo fora da raiz pública e impedir execução no armazenamento. Planejar retenção e limpeza, observando que exclusão lógica não libera espaço físico.

Critério de validação: ultrapassar a cota deve ser recusado sem deixar arquivos órfãos; downloads devem exigir autorização; arquivos não devem ser acessíveis por URL direta. Testar conteúdo suspeito somente com amostras de teste apropriadas em homologação.

6. Verificações necessárias no ambiente publicado

Estas são verificações pendentes, e não vulnerabilidades confirmadas:

• Confirmar a raiz pública e o bloqueio de acesso a `config`, `storage`, logs, backups, `.runtime`, `.git` e arquivos de ambiente. O caminho privado no código não comprova isolamento pelo servidor web.

• Conferir permissões mínimas da conta SQL e do usuário do serviço, versões suportadas dos componentes e atualização de dependências.

• Desativar exibição de erros em produção e restringir acesso aos logs. Manter alertas e rotação de registros.

• Executar backups protegidos do banco e dos documentos, com retenção definida e teste de restauração.

• Confirmar que contas e dados de demonstração não se confundem com cadastros reais e revisar os acessos administrativos.

• Validar autorização entre ONGs em ambiente controlado, inclusive leitura de documentos e alteração de registros por identificador.

7. Segurança e privacidade do PDF acadêmico

Na inspeção do arquivo de entrega não foram identificados anexos embutidos, JavaScript ou ações de execução. Os links encontrados são referências bibliográficas. A inspeção estrutural é limitada e não equivale a uma varredura antimalware completa.

O documento contém nomes e RGMs dos integrantes. Recomenda-se restringir a versão identificada ao contexto acadêmico e preparar cópia sem RGMs para publicação aberta. A exposição do modelo de tabelas, isoladamente, não caracteriza vulnerabilidade e não substitui a análise das permissões da aplicação.

Ordem sugerida de execução

1. Conferir a exposição de diretórios privados, o HTTPS e a configuração efetiva das sessões no servidor.

2. Tratar abuso do cadastro público e substituir a distribuição de senha inicial por ativação segura.

3. Aperfeiçoar a limitação de login e acrescentar cotas de upload.

4. Validar as mudanças em homologação, testar restauração de backup e planejar implantação com possibilidade de retorno à versão anterior.

O sistema já estar publicado exige avaliar compatibilidade e continuidade do serviço antes de alterar autenticação ou armazenamento. Nenhuma alteração de código, implantação, varredura remota ou correção de servidor foi executada como parte desta revisão documental.

Referências de segurança

• PHP. Securing Session INI Settings. Orientação sobre configuração de sessões e cookies.

• OWASP. File Upload Cheat Sheet. Controles complementares para envio e armazenamento de arquivos.

• OWASP. Cross-Site Request Forgery Prevention Cheat Sheet. Proteção contra requisições induzidas entre sites.

Fontes do projeto: `ong-connect/api/index.php`, `security.php`, `bootstrap.php` e `public-transfers.php`. A evidência é a cópia local revisada em 27/10/2026; correspondência com a implantação online não verificada.